Software Requirements Specification
Canteen Food Finder

23MCCE01 Pranav Prerepa , 23MCCE10 Piyush Kamal
March 3, 2026

1. Introduction

1.1 Purpose

This document provides a comprehensive description of the requirements for the Canteen Food Finder.
This document is intended for:

- Developers implementing the system
- Academic evaluators assessing the project
- Future maintainers of the system
- Stakeholders within the university

1.2 Scope

The Canteen Food Finder is a web-based information management system developed using the Django web framework and a relational database management system.
The primary objective of the system is to centralise and structure information related to university canteens, including:

- Food availability
- Pricing
- User-generated ratings
- Review moderation

The system provides:

1. Authenticated access for all users using university credentials.
2. Food browsing, searching, and comparison functionality.
3. Order placement and peer-based delivery mechanism.
4. Moderation controls for content validation.
5. Administrative tools for managing canteen, food, and user data.

The system is designed to operate within a university campus environment and is optimised for moderate user concurrency.The system includes a dynamic order and delivery workflow where users can place food orders and other users can accept and fulfill delivery requests. This introduces real-time interaction and state-based processing within the system.

1.3 Definitions, Acronyms, Abbreviations

SRS – Software Requirements Specification
UML – Unified Modeling Language
DBMS – Database Management System
ORM – Object Relational Mapping
MVT – Model-View-Template (Django architectural pattern)
HTTP – HyperText Transfer Protocol
Admin – System administrator with full control privileges
Moderator – Authorized user responsible for reviewing and moderating submitted content

1.4 References

- IEEE Recommended Practice for Software Requirements Specifications
- Django Official Documentation
- Python Programming Documentation
- Software Engineering Course Material

2. Overall Description

2.1 Product Perspective

The system is a standalone, server-based web application developed using Django, following the Model-View-Template (MVT) architectural pattern.
Architectural Components:
Presentation Layer

- HTML templates rendered via Django
- Client-side interactions via web browser

Application Layer

- Django views handling HTTP requests
- Business logic enforcement
- Role-based access control

Data Layer

- Relational database accessed via Django ORM
- Structured schema in Third Normal Form (3NF)
- Referential integrity enforced via foreign keys

Media Handling Subsystem

- Image validation
- File storage
- Image size and format enforcement

The system does not rely on micro-services, distributed computing, or third-party APIs. All functionality is internally managed.

2.2 Product Functions

The system shall provide the following major functional capabilities:

1. Browsing and Search
   - Retrieve list of all canteens
   - Retrieve food items alphabetically
   - Perform keyword-based search
   - Display price comparison across canteens
   - Filter results by price and rating
2. Authentication & Authorisation
   - Register user with university credentials
   - Authenticate users securely
   - Maintain session state
   - Enforce role-based permissions
3. Review Management
   - Submit review with rating and comment
   - Enforce one review per user per food per canteen
   - Allow editing and deletion of own review
   - Restrict image uploads to maximum three per review
   - Validate uploaded image size and type
4. Moderation Control
   - Approve or reject reviews
   - Remove inappropriate content
   - Maintain integrity of review system
5. Administrative Control
   - Add/Delete canteen
   - Add/Delete food items
   - Update menu pricing
   - Assign moderator roles
6. Order & Delivery Management
   - Place food orders with delivery location
   - View active and available delivery requests
   - Accept delivery requests by other users
   - Update delivery status (Accepted, Picked Up, Delivered, Completed)
   - Track order status by requesting user
   - Prevent users from accepting their own orders

2.3 User Characteristics

Registered User

- University student
- Basic web navigation skills
- Capable of submitting structured textual input
- Can act as both order requester and delivery partner
- Capable of handling order placement and delivery updates

Moderator

- Trusted user with content oversight responsibilities
- Familiar with system policies
- Responsible for content validation

Administrator

- Technically competent user
- Responsible for system configuration and data consistency

2.4 User Constraints

The system shall operate under the following user-related constraints:

- Only authenticated university users may submit reviews and upload images.
- Users must provide valid university credentials during registration.
- Users are limited to one review per food–canteen combination.
- A maximum of three images may be uploaded per review.
- Uploaded files must conform to supported image formats and size restrictions.
- Users must access the system through a web browser.
- Users cannot accept delivery requests created by themselves.
- Each order can be assigned to only one delivery partner.
- Order status must follow predefined transitions (Placed → Accepted → Picked Up → Delivered → Completed).

2.5 Assumptions and Dependencies

- All users have internet access.
- Django runtime environment is correctly configured.
- Database server remains operational.
- University authentication information is valid.
- Image storage directory is accessible and writable.

The system depends on:

- Python runtime environment
- Django framework
- Relational database management system
- Web browser compatibility

2.6 Apportioning Requirements

The following requirements are identified for potential future implementation and are outside the scope of the current academic version:

- Mobile application version of the system
- Real-time canteen availability tracking
- Location of various vending machines, shops as well as availability of products
- Integration with external university authentication systems
- Integration of digital payment systems for automated transaction handling
- Real-time delivery tracking system

These enhancements may be implemented in future system iterations

3. Specific Requirements

3.1 Interface Requirements

3.1.1 External Interface Requirements

The system shall provide a web-based graphical user interface accessible through standard web browsers.
The user interface shall:

- Provide navigation menus for browsing canteens and food items.
- Provide a search bar for keyword-based food search.
- Display tabular and structured representations of:
  - Canteen names
  - Menu listings
  - Food prices
  - Ratings
- Provide secure login and registration forms.
- Provide controlled image upload interface for authenticated users.
- Provide moderation and administrative dashboards for authorised roles.
- Interface for placing food orders
- Interface for viewing available delivery requests
- Interface for accepting and updating delivery status
- Dashboard for tracking order progress

All UI elements shall be rendered using Django templates and shall ensure consistency across pages.
The system shall support responsive design principles to allow usability across desktop and mobile browsers.

3.1.2 Hardware Interface Requirements

The system does not require specialized hardware components.
Minimum hardware requirements include:

Client-Side:
- Device capable of running a modern web browser.
- Internet connectivity.

Server-Side:
- Server capable of running Python and Django runtime environment.
- Persistent storage for database and media files.
- No direct hardware device integration is required.

3.1.3 Software Interface Requirements

The system shall interface with the following software components:

- Django Web Framework
- Handles routing, authentication, and ORM operations.
- PostgreSQL / MySQL / SQLite
- Accessed via Django ORM.
- Compatible with Windows, Linux, or macOS environments supporting Python runtime.
- Media File Storage
- Local file system or configured storage backend for uploaded images.
- Web Browser. Google Chrome, Mozilla Firefox, Microsoft Edge (latest stable versions recommended).

No third-party external APIs are required for core functionality.

3.1.4 Communication Interface Requirements

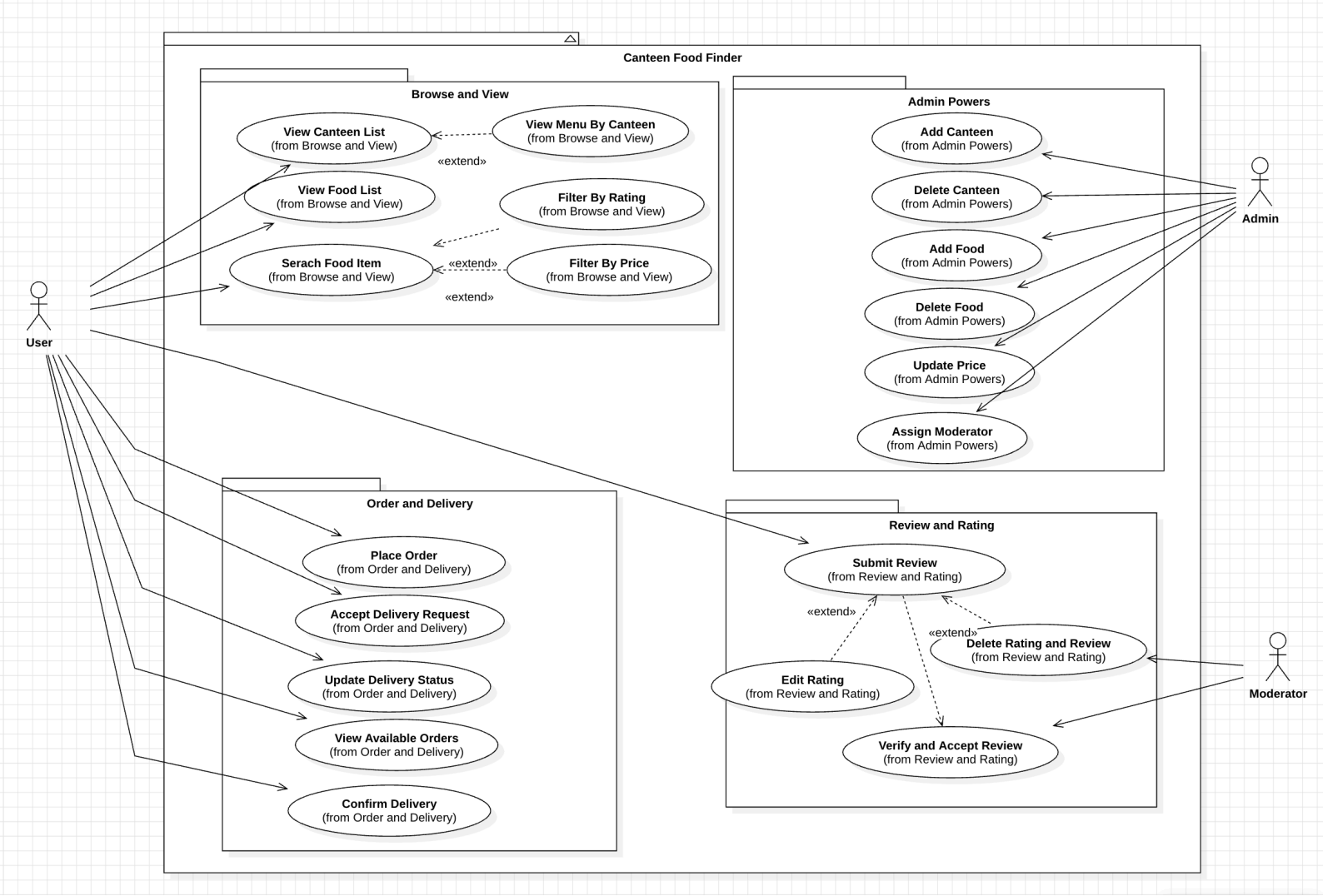
The system shall communicate using standard HTTP/HTTPS protocols.
Communication characteristics include:

Client-server architecture.
Stateless HTTP request-response model.
Secure communication via HTTPS (if deployed).
Session-based authentication management.

The system shall support:

LAN-based deployment within university network.
Internet-based deployment for broader accessibility.

3.2 Functional Requirements



3.3 Performance Requirements

- The system shall respond to search queries within 2 seconds under normal load.
- Image upload shall be processed within acceptable network latency.
- The system shall support concurrent access by campus-scale users.
- The system shall handle concurrent order placement and delivery requests efficiently.
- Order status updates shall be reflected in near real-time for users.

3.4 Logical Database Requirements

The database shall include the following normalised entities:

- Canteen
- Food
- User
- Menu
- Review
- FoodImage
- Order

Constraints:

- Composite primary keys where applicable.
- Foreign key enforcement.
- Unique constraints on user registration number and email.
- Rating restricted to integer range 1–5.
- Maximum three image records per (user_id, food_id, canteen_id).
- Order status shall follow defined state transitions
- Each order must reference a valid user, food item, and canteen
- Delivery partner must be different from ordering user

The schema shall conform to Third Normal Form (3NF).

3.5 Design Constraints

- The system shall be developed using the Django web framework and Python programming language.
- The application shall follow the Model–View–Template (MVT) architectural pattern.
- The database schema shall maintain normalisation (minimum Third Normal Form) and enforce referential integrity using foreign keys.
- Role-based access control shall restrict review submission, moderation, and administrative actions.
- User passwords shall be securely hashed and not stored in plain text.
- Uploaded images shall be limited in number and validated for format and size.
- The system shall operate as a web-based application accessible through standard web browsers.
- The scope of the system shall remain limited to academic project requirements and campus-scale deployment.
- The system shall implement a state-based workflow for order and delivery management.
- All system operations shall require authenticated user access.

•

3.6 Software System Attributes

Reliability

The system shall maintain database consistency and prevent invalid state transitions, particularly in the order and delivery lifecycle.

Availability

The system shall be accessible during operational hours with minimal downtime.

Security

- Passwords shall be hashed using secure cryptographic hashing.
- Role-based authorisation shall restrict privileged operations.

Maintainability

The system shall follow modular design principles to allow future expansion.

Portability

The system shall be deployable on platforms supporting Python and Django.